

NexiLock

Rapport d'activités - Présentation de mon projet

BTS SIO option SLAM - Emma CLUZET - CFA Doranco - Session 2026

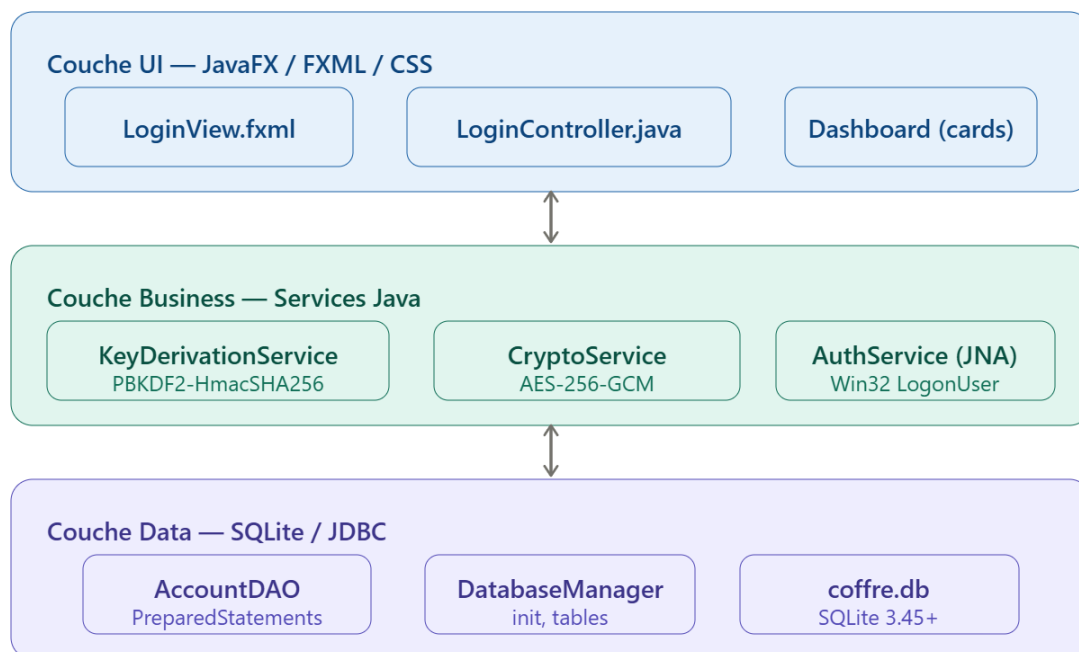
Présentation du projet

Pour mon épreuve E5, j'ai développé NexiLock, un gestionnaire de mots de passe desktop. L'idée de départ est simple : beaucoup de gens réutilisent les mêmes mots de passe parce que c'est difficile d'en retenir des dizaines. NexiLock propose de tout stocker dans un fichier chiffré sur l'ordinateur, sans passer par le cloud.

Le projet a été lancé dans un cadre de groupe en début de formation. Mes camarades se sont finalement retirés du projet en cours de route, et j'ai décidé de le continuer seule jusqu'à la livraison finale. C'est donc un travail que j'ai réalisé intégralement en autonomie, de la conception à la livraison.

Élément	Détail
Établissement	CFA Doranco Espace Multimédia, Paris 10e
Période	Février - mars 2026, soit 4 semaines de travail
Organisation	Méthode Agile Scrum simplifiée : 4 sprints d'une semaine chacun
Dépôt Git	https://github.com/emmacluzet/NexiLock
Version livrée	NexiLock v1.0.0 - tag Git release/1.0.0

Présentation du projet	1
Ce que fait l'application	1
Les technologies que j'ai utilisées	1
Organisation de mon travail : les 4 sprints.....	2
Sprint 1 - 3 au 7 février : mettre en place les fondations	2
Sprint 2 - 10 au 14 février : sécuriser les données	2
Sprint 3 - 17 au 21 février : construire l'interface principale.....	2
Sprint 4 - 24 au 28 février : tester, corriger et livrer	2
Les livrables de mon projet	2
Ce que j'ai appris.....	3
Côté technique	3
Côté méthode.....	3



Ce que fait l'application

NexiLock est une application Java avec une interface graphique (JavaFX). Voici ce que l'utilisateur peut faire :

- Se connecter avec son mot de passe Windows : l'application vérifie l'identité via une API Windows native (Win32 LogonUser, accessible avec la bibliothèque JNA).
- Déverrouiller le coffre avec une clé de secours (Recovery Key) si le mot de passe Windows a changé.
- Ajouter, modifier, supprimer et rechercher des identifiants (service, login, mot de passe).
- Générer automatiquement des mots de passe robustes entre 8 et 32 caractères.
- Tout stocker localement dans un fichier SQLite (coffre.db), les mots de passe étant chiffrés en AES-256-GCM.

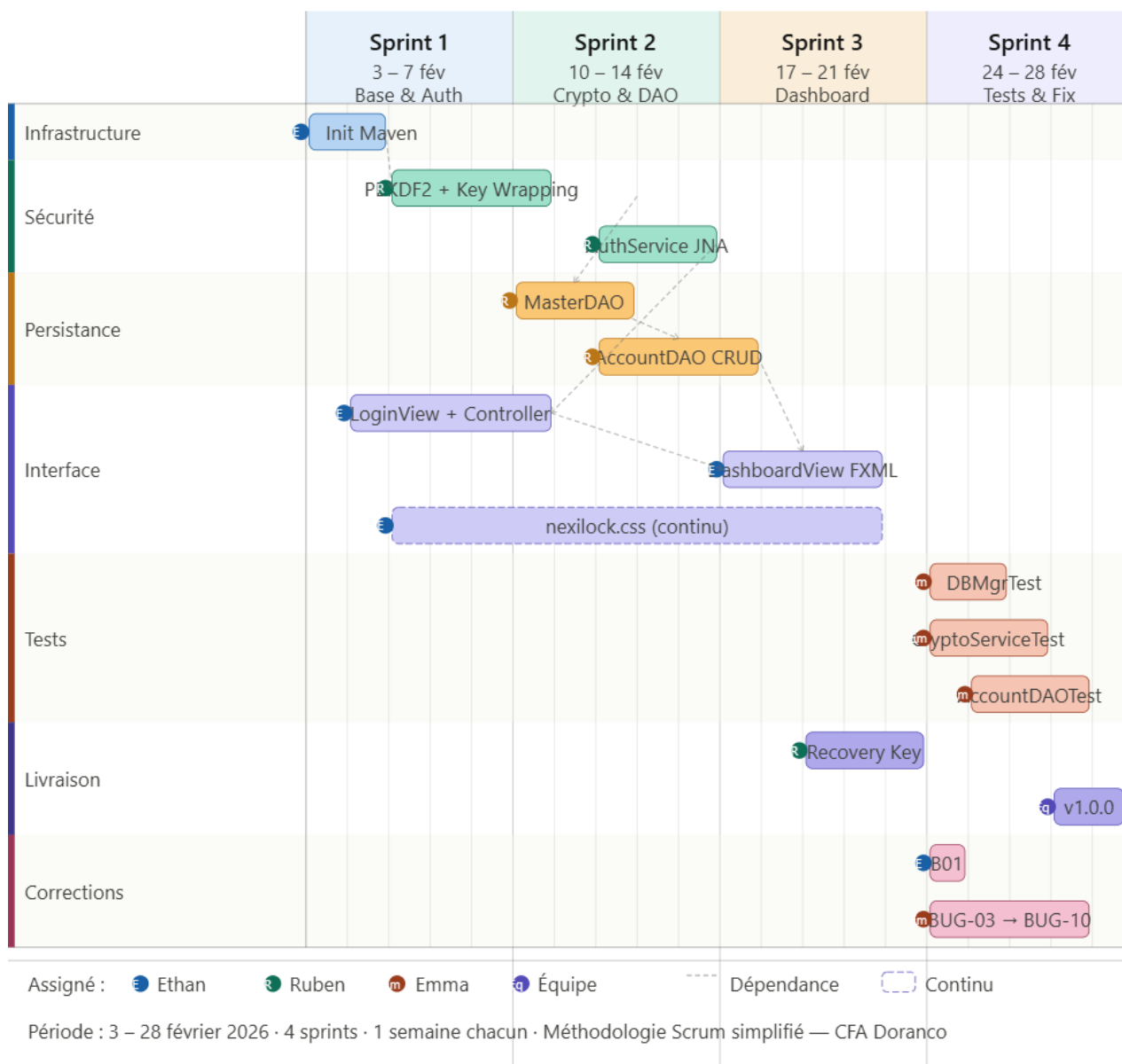
L'application ne se connecte jamais à internet. Toutes les données restent sur l'ordinateur de l'utilisateur.

Les technologies que j'ai utilisées

J'avais déjà vu Java en cours, mais plusieurs éléments de ce projet étaient nouveaux pour moi. Voici ce que j'ai dû apprendre ou approfondir :

Technologie	Utilisation dans NexiLock	Nouveauté pour moi
Java 21	Langage principal	Non : déjà vu en cours
JavaFX 21 + FXML	Interface graphique (fenêtres, boutons, formulaires)	Oui : première utilisation
SQLite via JDBC	Base de données locale pour stocker les comptes	Partiellement : SQL connu, JDBC nouveau
PBKDF2-HmacSHA256	Dériver une clé secrète à partir du mot de passe	Oui : notion de cryptographie apprise ici
AES-256-GCM	Chiffrer les mots de passe stockés	Oui : premier chiffrement symétrique
JNA (Java Native Access)	Appeler l'API Windows pour vérifier le mot de passe	Oui : première fois que j'appelle du code natif

Maven	Gestion des dépendances et compilation	Partiellement : vu en cours mais approfondi
JUnit 5	Tests unitaires	Partiellement : tests simples déjà vus
Git / GitHub	Gestion de versions, branches, issues	Non - utilisé depuis le début de la formation



Organisation de mon travail : les 4 sprints

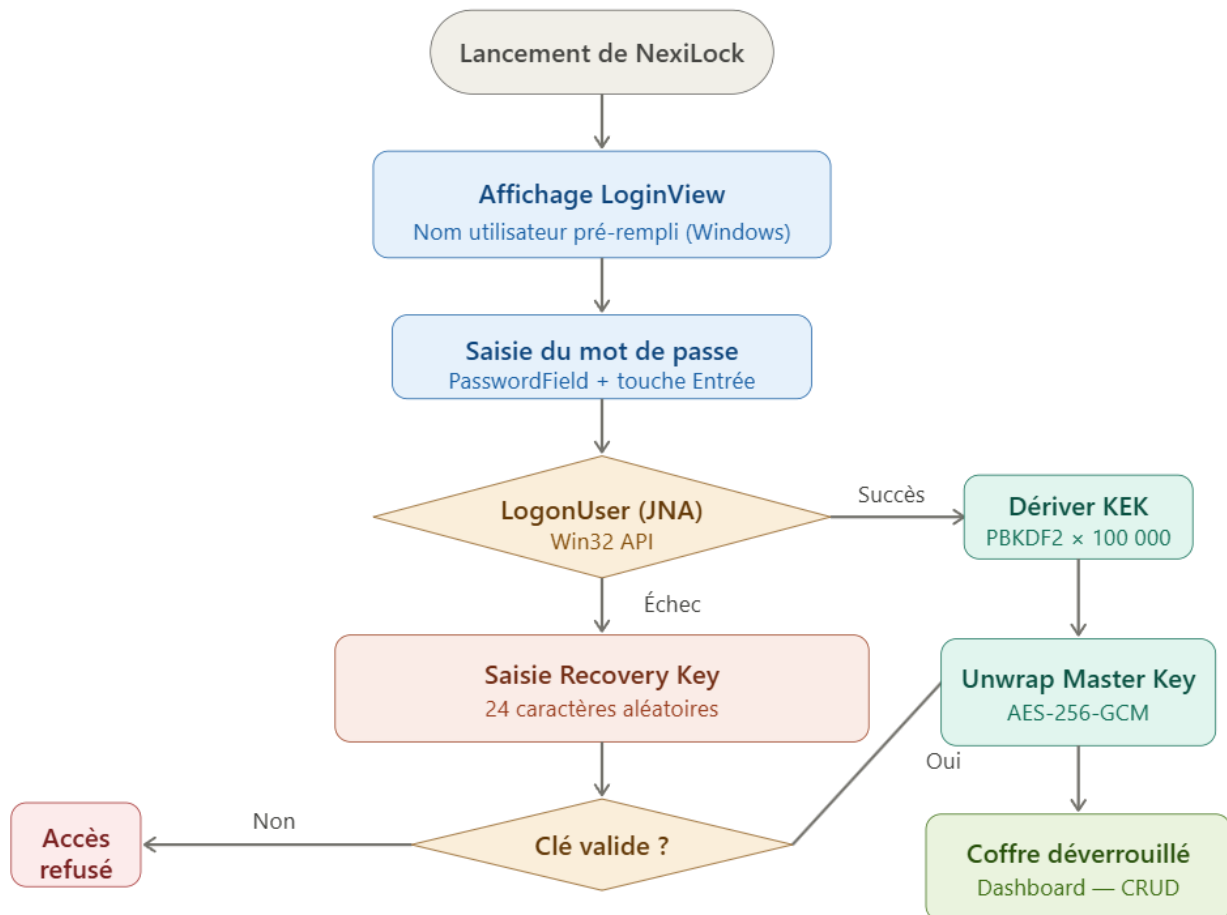
J'ai organisé mon travail en 4 sprints d'une semaine, chacun avec un objectif précis. J'ai suivi mon avancement via les issues GitHub que j'ai créées moi-même.

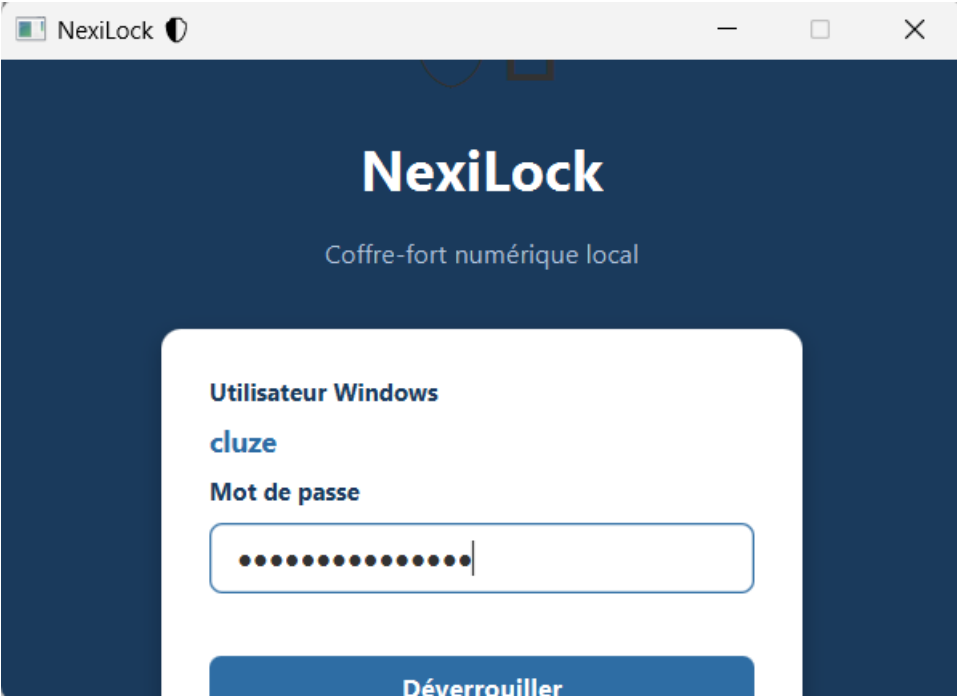
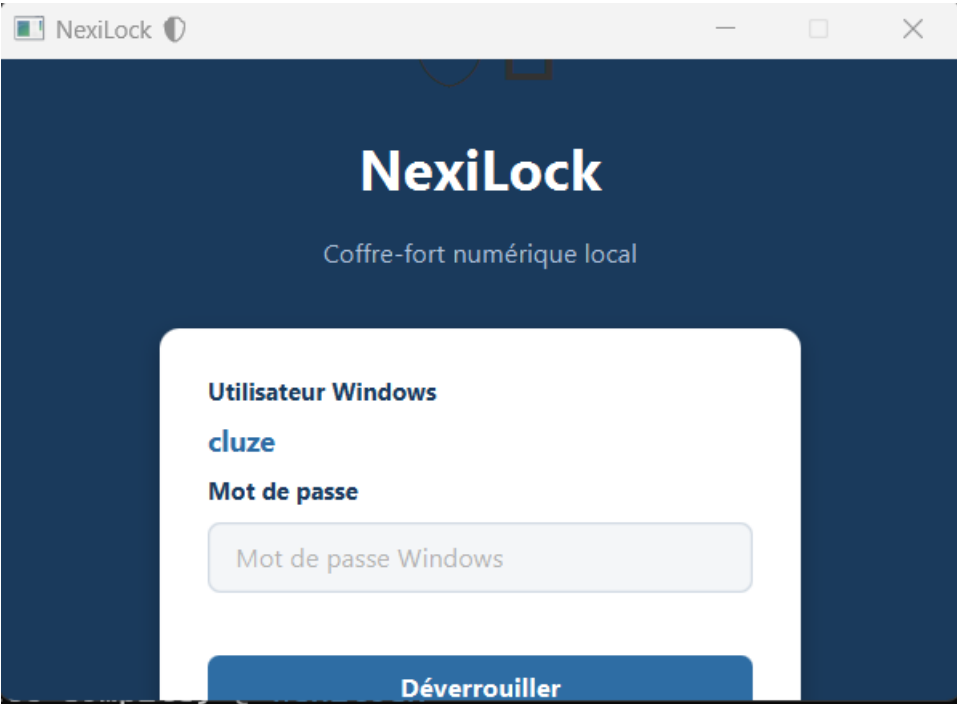
Sprint 1 - 3 au 7 février : mettre en place les fondations

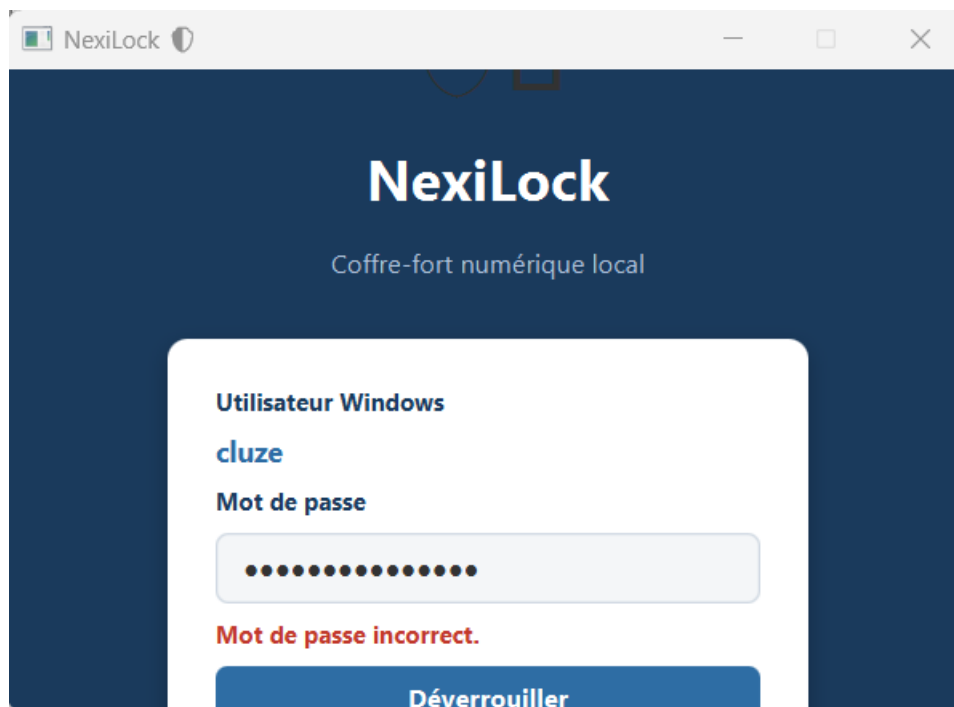
L'objectif de ce sprint était d'avoir une base fonctionnelle : le projet Maven configuré, la base de données créée et l'interface de connexion affichée à l'écran.

- J'ai créé le projet Maven avec toutes les dépendances nécessaires dans pom.xml.

- J'ai écrit la classe DatabaseManager qui crée automatiquement le fichier coffre.db et les deux tables (master et accounts) au premier lancement.
- J'ai développé le service d'authentification Windows (AuthService) en utilisant JNA pour appeler LogonUser, l'API Win32 qui vérifie un mot de passe Windows.
- J'ai créé la fenêtre de connexion (LoginView.fxml + LoginController) avec le nom d'utilisateur Windows pré-rempli, le champ mot de passe et le bouton de déverrouillage.
- J'ai commencé la feuille de styles CSS pour donner une apparence propre à l'application.







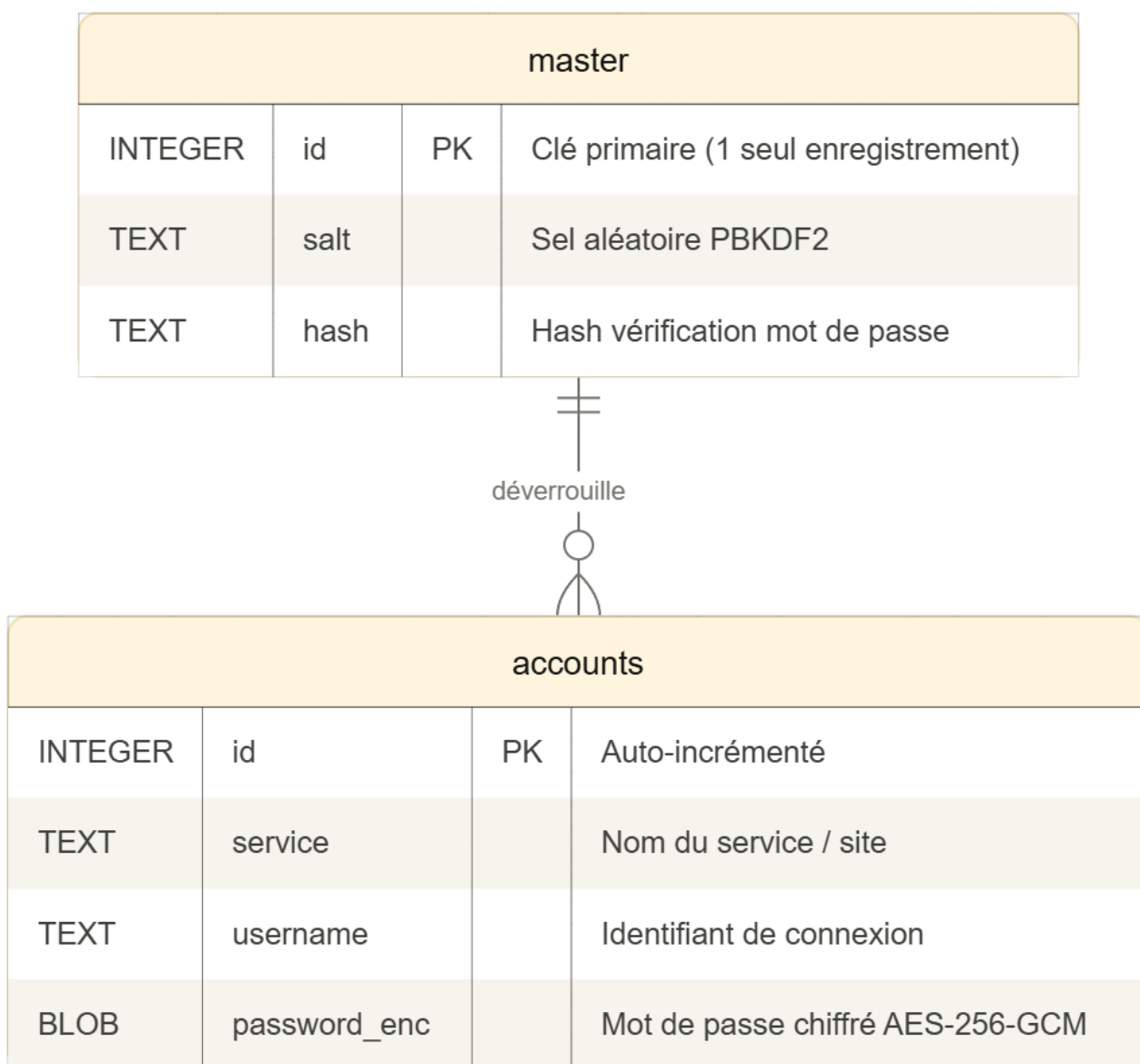
Sprint 2 - 10 au 14 février : sécuriser les données

Ce sprint était le plus complexe techniquement. Il fallait mettre en place toute la partie cryptographique.

J'ai développé CryptoService, le cœur de la sécurité. Il utilise PBKDF2 avec 100 000 itérations pour dériver une clé KEK à partir du mot de passe. Cette KEK sert à chiffrer ("wrapper") la Master Key qui, elle, chiffre les mots de passe des comptes.

J'ai créé MasterDAO pour stocker en base le sel (salt), le hash de vérification et la Master Key chiffrée.

J'ai développé AccountDAO avec les 4 opérations CRUD. Toutes les requêtes utilisent des PreparedStatements pour éviter les injections SQL. Les mots de passe sont chiffrés avec AES-256-GCM avant d'être sauvegardés.



Sprint 3 - 17 au 21 février : construire l'interface principale

Ce sprint était centré sur l'interface utilisateur et les fonctionnalités visibles.

J'ai créé le tableau de bord (DashboardView.fxml + DashboardController) avec la liste des comptes sous forme de cartes, le formulaire CRUD et la barre de recherche en temps réel.

J'ai ajouté le générateur de mots de passe avec un slider pour choisir la longueur (8 à 32 caractères).

J'ai implémenté la Recovery Key : au premier lancement, une clé de 24 caractères est générée et affichée. Elle permet de déverrouiller le coffre si le mot de passe Windows change.

J'ai finalisé le CSS pour les deux écrans (connexion et tableau de bord).

The screenshot shows the NexiLock web application interface. At the top, there's a dark blue header with the NexiLock logo and a search bar labeled "Rechercher un service...". Below the header, on the left, is a sidebar with the text "Mes comptes". The main content area is titled "Service" and contains a form for creating a new account. The form has the following fields: "Service" (with a placeholder "ex: GitHub, Gmail..."), "Identifiant" (with a placeholder "email ou nom d'utilisateur"), and "Mot de passe" (with a placeholder "Mot de passe" and a toggle icon). Below the password field is a "Générateur de mot de passe" section with a slider for "Longueur" ranging from 8 to 19, and a "Générer" button. At the bottom of the form are two buttons: "Enregistrer" (blue) and "Supprimer" (red).

Sprint 4 - 24 au 28 février : tester, corriger et livrer

Ce sprint était dédié à la qualité : tests, corrections des bugs et documentation.

J'ai refactorisé DatabaseManagerTest pour utiliser une base temporaire isolée (les tests ne touchent plus au vrai coffre.db).

J'ai écrit CryptoServiceTest (15 tests) qui vérifie tous les cas du moteur cryptographique.

J'ai écrit AccountDAOTest (6 tests) qui vérifie le CRUD complet avec chiffrement réel.

J'ai corrigé 10 bugs identifiés lors des tests (voir le backlog de corrections).

J'ai rédigé tous les documents de ce dossier et créé le tag v1.0.0 sur GitHub.

Les livrables de mon projet

Livrable	Format	Sprint
Code source Java (14 classes)	Fichiers .java	Sprints 1 à 4
Interface graphique (FXML + CSS)	Fichiers .fxml et .css	Sprints 1 et 3
Suite de tests JUnit 5 (27 tests)	Fichiers .java	Sprints 2 et 4
Cahier des charges	Document Word	Sprint 1
Backlog de correction (10 tickets)	Document Word	Sprint 4
Plan de branches Git et commits	Document Word	Sprint 4
Plan de tests d'intégration	Document Word	Sprint 3
Rapport de tests (résultats)	Document Word	Sprint 4
Procédure d'installation	Document Word	Sprint 4
Compte rendu d'interventions	Document Word	Sprint 4
Tableau de bord de suivi de projet	Document Word	Sprint 4
Rapport d'activités (ce document)	Document Word	Sprint 4

Ce que j'ai appris

Ce projet m'a beaucoup appris, notamment sur des sujets que je n'avais jamais abordés en cours.

Côté technique

La cryptographie appliquée : comprendre la différence entre hachage et chiffrement, et pourquoi il faut 100 000 itérations PBKDF2 pour résister à la force brute, c'était nouveau pour moi.

Le Key Wrapping : l'idée de chiffrer une clé avec une autre clé pour ne jamais exposer la Master Key en base de données m'a pris du temps à comprendre, mais c'est une technique standard en sécurité.

JavaFX : apprendre FXML et la liaison Controller/View en parallèle du reste du projet était un challenge.

JDBC et SQLite : gérer les connexions proprement (try-with-resources) et utiliser des PreparedStatements pour éviter les injections SQL.

Côté méthode

Travailler seule en Scrum m'a appris à découper le travail en petites tâches réalistes et à ne pas tout vouloir faire en même temps.

Les tests unitaires m'ont aidée à trouver des bugs que je n'avais pas vus manuellement (notamment le bug BUG-05 qui faisait que les tests écrivaient dans le vrai coffre.db).

Git avec des branches par fonctionnalité, c'est beaucoup plus clair qu'un seul main avec tout dedans.